

Closing the Loop: Spec-Grounded, Self-Verifying AI Authoring of X3D 4.0 Scenes in the Browser

Documentation for the *X3D Copilot* prototype

Alberto Jaspe-Villanueva, King Abdullah University of Science and Technology (KAUST)
alberto.jaspe@kaust.edu.sa

Prototype: <https://albertojaspe.net/x3d-copilot/> Source (MIT): <https://github.com/ajaspe/x3d-copilot>
Web3D 2026, AI & Web3D Innovation Competition. Category: AI-Driven Content Generation.

Trying it: the AI features need the user's own Gemini or Claude API key, entered once in **Settings** (a free Gemini key from aistudio.google.com/apikey is enough). The key stays in the browser and is sent only to the model provider. Everything else, editing, validation, direct manipulation, import and export, works without a key.

X3D Copilot is a web application for creating and editing 3D scenes in X3D, the ISO-standard open format for 3D on the web, in its latest version, X3D 4.0. It combines a conventional editor (live rendering, source view, direct manipulation of objects, import and export of common 3D formats) with an AI co-author to which the user can describe a scene, or a change to a scene, in plain language. The novelty is in how the AI is connected to the editor. Language models produce X3D quickly, but on their own they invent field names, place nodes in the wrong parents, wire events incorrectly and never see the result. In X3D Copilot the model reads node definitions from the official X3D specification data, can change the scene only through edits that are immediately checked by three independent validators (the official X3D schema, a semantic linter derived from the specification, and the X_ITE runtime), receives those reports back until the scene is valid, and finally looks at a screenshot of the rendered result. Scenes produced this way conform to the standard by construction. The tool runs entirely in the browser as an open-source static site; the only network traffic is the call to the model, made with the user's own key.

What the tool does

The interface (Figure 1) has three main panes and works like a normal editor even with the AI switched off.

- **Describe a scene, or a change, in a sentence.** “A snowman on snowy ground at night, under a starry sky.” “Give him a carrot nose, coal eyes and a top hat.” “Make the moonlight cast shadows.” “When I click the snowman, his hat should do a small jump.” Each of these was a real prompt; together they produced the scene in Figure 1, which now ships as an example. The user watches the model work step by step (read the scene, look up a node, apply an edit, validate, take a screenshot) and can expand any step to see exactly what the model received.
- **Repair broken scenes.** A deliberately broken example with eleven planted errors (misspelled profile, old version, a material outside its appearance, a wrong field name, a bad ROUTE, mismatched interpolator keys, an undefined reference, and more) is fixed by the model in one request, with every fix listed.
- **See the scene live.** The X_ITE X3D browser renders the scene as the source changes, including animation, sensors, physically based materials, shadows and HAnim characters, with shading modes and screenshots one click away.
- **Edit the source with help.** The code editor offers autocompletion generated from the X3D specification: node names, the children each node accepts, and every field with its type and default value. Problems are shown inline and in an issue list with line numbers, explanations and suggested fixes.
- **Move things by hand.** Clicking an object selects it and shows a gizmo for moving, rotating and scaling, plus a numeric inspector. Changes are written back into the source as minimal edits, and the selection is passed to the AI, so “make this twice as big” refers to the object the user clicked. The gizmo itself is made of standard X3D sensor nodes.
- **Bring in other formats, take out X3D.** glTF, OBJ, STL, PLY, SVG and VRML files import as editable X3D 4.0 source; scenes export as X3D XML, Classic VRML, JSON or PNG.
- **Nothing to install, nothing sent to a server.** A static site; the model provider is the only external service, and the user chooses it (Google Gemini or Anthropic Claude) and pays for it with their own key.



Figure 1: The interface, with the copilot-authored snowman example loaded and its hat selected. (1) AI co-author: the conversation with the model; each tool call appears as a step that can be expanded to show the validation report the model received. (2) Live 3D view: X_ITE renders the scene as the source changes; clicking geometry selects it. (3) X3D source: a code editor with specification-driven autocompletion and inline diagnostics. (4) Validation: findings of the three validators, with jump-to-line. (5) Inspector: translation, rotation and scale of the selected object, editable, synchronised with the source and passed to the model as context. (6) Toolbar: examples, import, export, shading, view control and model settings.

Technical overview

Grounding the model in the specification

The Web3D Consortium publishes the X3D Unified Object Model (X3DUOM), a machine-readable description of every node in the X3D 4.0 specification. A build script compiles it into a compact database of 260 concrete nodes, 78 abstract types and 17 statements, each with its fields, types, access types, defaults, accepted child types and default container field. This database drives three things: the editor’s autocompletion, the semantic linter, and the model’s `lookup_node` and `search_nodes` tools. When the model is unsure how a node is defined, it reads the specification instead of guessing.

The verification loop

The model never writes to the scene directly. It receives seven tools: `get_scene`; `edit_scene` (exact, unique text replacements or anchored insertions; ambiguous edits are rejected); `replace_scene`; `validate_scene`; the two lookup tools; and `screenshot`, which returns the current frame as an image. Every scene-modifying tool answers with a validation report from three independent layers:

1. **Schema.** The official `x3d-4.0.xsd`, with the Web3D extension schemas and the W3C XML-Signature schema it imports, is compiled once by `libxml2` running in WebAssembly; a typical scene validates in a few milliseconds.
2. **Semantic linter.** About thirty rules that a schema cannot express: duplicate DEF, undefined USE, USE with a different node type, ROUTE to an unknown node or field, ROUTE from a non-output or to a non-input field, ROUTE type mismatch, a node in a parent field that does not accept it, several children in a single-node field, attribute arity, type and range checks, zero rotation axes, interpolator key and keyValue consistency, empty groups, shapes without geometry, missing viewpoint, and unknown nodes or fields with suggestions.
3. **Runtime.** X_ITE parse errors and console warnings captured while the scene loads.

Reports are graded (error, warning, hint), attributed to their source and phrased as actions. Schema messages that duplicate a linter finding on the same line are demoted so the model is not penalised twice. The agent loop repeats until the model stops calling tools, up to 24 rounds per request. The system prompt is short; it fixes the workflow (read before editing, prefer patches, resolve errors before answering, look up when unsure, screenshot when appearance matters) and a compact X3D 4.0 authoring guide. Correctness is enforced by the tools, not by prompt text.

Direct manipulation as shared context

After each load, invisible `TouchSensor` nodes are injected into every `Transform` through the Scene Access Interface, so clicking geometry selects its innermost transform. The gizmo is a group of `PlaneSensor` and `CylinderSensor` nodes inserted next to the selected node in its parent’s frame; a `ProximitySensor` tracks the camera so the gizmo keeps a constant screen size. Drags are written back to the source as minimal attribute

edits through the editor's syntax tree, and the current selection (DEF name, line, transform) is appended to the user's message. These runtime nodes never enter the user's file.

Engineering

TypeScript and Vite; 56 automated tests (schema fixtures, one test per linter rule, edit patching, gizmo mathematics, and a check that every shipped example is lint-clean and schema-valid); continuous integration on GitHub; production build of about 3 MB of application code plus the vendored X_ITE assets, hosted as static files.

Architecture

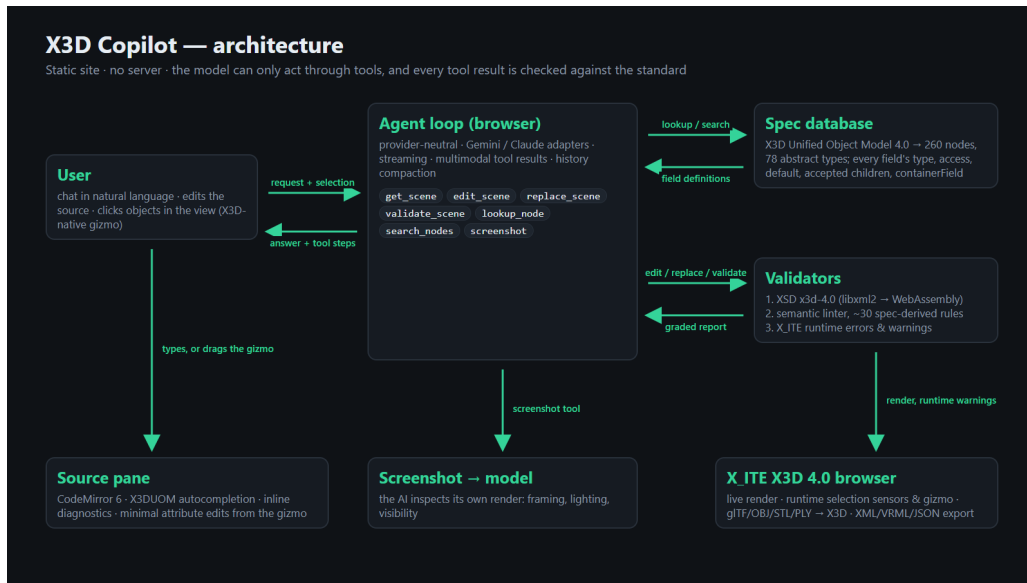


Figure 2: Everything runs in the browser. The agent loop reaches the scene only through tools; the specification database answers lookups; the validators and the X_ITE runtime grade every change; screenshots return the render to the model. Provider adapters keep the loop independent of the model vendor.

AI models used

Google Gemini 3.8 Flash (default), Gemini 3.1 Pro and Gemini 2.5 Pro/Flash; Anthropic Claude Opus 5, Sonnet 5 and Haiku 4.5. All are accessed through the vendors' official JavaScript SDKs directly from the browser, with streaming, native function calling, images inside tool results (so the model can see its render) and the vendor's effort or thinking controls. A provider interface (`runTurn`, `listModels`, `describeError`) isolates each SDK in an adapter of about 150 lines; adding a provider is one file. The user selects provider and model in the settings and can fetch the list of models their key allows.

Web3D frameworks employed

- **X3D 4.0**, the current version of the standard, with its XML encoding; scenes are plain `.x3d` files.
- **X3D Unified Object Model 4.0** (Web3D Consortium), compiled into the specification database described above.
- **Official X3D 4.0 XML Schema**, executed in the browser through libxml2 compiled to WebAssembly.
- **X_ITE 16.3**, the open-source X3D 4.0 browser, as rendering and runtime engine, and as the converter behind import (glTF, OBJ, STL, PLY, SVG, VRML) and export (XML, Classic VRML, JSON).
- **CodeMirror 6** for the editor, with completions generated from the object model.

How the prototype addresses the competition criteria

Innovation and originality

To our knowledge, this is the first X3D authoring tool in which a language model reads node definitions from the specification through a tool rather than from training memory, can act only through validated edits whose reports are returned to it, and inspects a screenshot of its own render before answering; and the first browser X3D editor whose transform gizmo is built from X3D sensor nodes. The individual techniques are known; the contribution is their combination into a closed loop that makes AI-generated X3D conform to the standard by construction.

Technical excellence

Three independent validators sharing one issue format; a specification database regenerated from the official X3DUOM; provider-neutral agent loop with typed adapters; direct manipulation implemented with standard X3D nodes and synchronised with the source through the editor’s syntax tree; 56 automated tests and continuous integration; a static build with no server-side component.

AI integration

The AI is not a chat window next to an editor: it is wired into the editor’s own validation and rendering. Its tools give it the specification, the validators and its own eyes, and its steps are visible and inspectable to the user. Two vendors and six models are supported through the same loop, chosen by the user.

User experience

No installation; one URL. Plain-language authoring alongside conventional editing, with the model’s work shown step by step. Direct manipulation with a gizmo and inspector; errors phrased as actions with suggestions and jump-to-line; eight example scenes including one built entirely through the copilot and one deliberately broken for trying the repair loop. Validation of a typical scene takes milliseconds; rendering is X_ITE’s; only the model calls take seconds.

Impact and scalability

For the Web3D community the tool lowers the entry barrier to X3D to a sentence in a chat box while raising conformance, which is relevant to teaching, rapid prototyping and the adoption of the standard. The pattern (machine-readable specification, validators as tools, visual check) transfers to glTF, USD or CityGML by exchanging the schema and object model. Deployment scales trivially because the compute is the user’s browser and their own model account. The project is open source under the MIT license.

Innovation statement

Large language models can write 3D scene descriptions, but in practice their X3D is unreliable: they invent field names, confuse versions, misplace nodes and never see what they produced. The usual answer is a bigger model or more prompt text. X3D Copilot takes a different route: it grounds the model in the standard itself and closes the loop with verification the model cannot bypass.

What is new. (1) *Specification-grounded generation.* The X3D Unified Object Model is compiled into a queryable database (260 nodes, every field with type, access, default, accepted children and container field). The model reaches it through a lookup tool, so when it is unsure about a field or where a node may go, it reads the specification instead of guessing. (2) *Closed-loop, multi-layer verification.* The model can only change the scene through tools, and every change returns a report from three independent validators: the official X3D 4.0 XML Schema running as libxml2 in WebAssembly, a semantic linter derived from the object model, and the X_ITE runtime. The model repairs its own errors until the report is clean. (3) *The AI looks at its work.* A screenshot tool returns the rendered frame, so the model checks framing, lighting and visibility, which no schema can express. (4) *Shared spatial context.* Users select objects in the 3D view with a gizmo built from X3D sensor nodes; the selection is passed to the model, so “rotate this” resolves to an exact node. (5) *Model-agnostic and in-browser.* The agent loop is provider-neutral (Gemini and Claude today), runs entirely client-side with the user’s key, and needs no server.

Why it matters. The result is AI-generated content that is standards-conformant by construction: every scene that leaves the tool passes the official schema and renders in a conformant browser. In our recorded sessions the model repairs a scene with eleven planted errors in a single request and builds complete animated, interactive scenes (nested orbits, sensors, physically based materials, shadows, click behaviours) from short prompts, verifying them visually before answering; those scenes ship as examples.

Impact and scalability. The approach is not specific to X3D. The specification database is generated by a script, and the same architecture applies to glTF, USD or CityGML with a different schema and object model. For the Web3D community it lowers the entry barrier to X3D to a sentence in a chat box while raising conformance, which helps education, rapid prototyping and the long-term health of the standard. Everything is open source, covered by automated tests, and deployed as a plain static site that anyone can host. (*About 420 words.*)

Limitations and next steps

Selection targets **Transform** nodes only; prototype instances are not selectable. The screenshot check is qualitative and the model’s aesthetic judgement varies between runs. Long generations depend on provider speed and quotas. Planned work: HAnim-aware tools (joint lookup, motion import), glTF export through X_ITE, retrieval over the X3D examples archive, and a quantitative evaluation of first-pass validity with and without the specification tools.